Detectarea și Recunoașterea Facială a Personajelor din Scooby-Doo

Dragomir Daria Nicoleta, grupa 352

1 Introducere

Scopul acestui proiect este detectarea și recunoașterea facială a personajelor din serialul de desene animate *Scooby-Doo* (Task 1 și Task 2). Abordarea aleasă se bazează pe paradigma **Sliding Window** (fereastră glisantă) combinată cu o arhitectură de rețele neuronale convoluționale (**CNN**) antrenată specific pentru fiecare clasă.

Strategia generală a fost antrenarea a 5 clasificatoare binare distincte (câte unul pentru Fred, Daphne, Velma, Shaggy și un model generic pentru clasa "Unknown"), urmând ca rezultatele acestora să fie agregate pentru rezolvarea celor două task-uri.

2 Metodologie și Arhitectură

2.1 Arhitectura Modelului (CNN)

Pentru clasificarea ferestrelor extrase din imagini, am proiectat o rețea convoluțională personalizată cu 5 blocuri convoluționale. Această arhitectură a fost replicată pentru fiecare dintre cele 5 modele, fiind antrenată independent pe setul de date specific fiecărui personaj.

Arhitectura rețelei este definită astfel:

- **Input:** Imagine RGB redimensionată la dimensiunea ferestrei specifice personajului.
- **Bloc 1-4:** `Conv2d` (kernel 3×3, padding 1) $\rightarrow$ `BatchNorm2d` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2d` (kernel 2, stride 2). Numărul de canale crește progresiv: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$.
- **Bloc 5:** `Conv2d` ($256 \rightarrow 512$ canale) $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU` $\rightarrow$ `AdaptiveAvgPool2d(1,1)`.
- **Output:** Strat `Flatten` urmat de un strat `Linear` ($512 \rightarrow 1$) care produce un logit pentru clasificarea binară.

2.2 Antrenarea Modelelor

Modelele au fost antrenate folosind o abordare de tip **One-vs-Rest**.

Augmentarea Datelor: Pentru a crește robustețea modelului, setul de antrenare a fost îmbogățit prin:

- Adăugarea fețelor *zoomed-out* (micșorate) pentru a ajuta detectorul să recunoască fețe la scări diferite.
- *Flipping* orizontal (oglindire).
- Folosirea de exemple negative extrase din fundal (background patches) și fețe ale altor personaje marcate ca negative.

Parametri de antrenare:

- **Optimizator:** Adam cu o rată de învățare (lr) de $1e-5$.

- **Funcție de Cost:** `sigmoid_focal_loss`. Aceasta a fost preferată în locul *BCEWithLogitsLoss* standard pentru a gestiona dezechilibrul major dintre clase (mult mai multe exemple negative decât pozitive) și pentru a forța modelul să se concentreze pe exemplele "dificile".
- **Metrică:** Scorul F1 a fost monitorizat pentru a salva cel mai bun model (*checkpoints*).
- **Epochs:** 35 epoci per model.

3 Algoritmul de Detecție (Inference)

Procesul de inferență se realizează prin scanarea imaginii folosind ferestre glisante la multiple scale.

3.1 Parametrii Sliding Window

Detectarea se bazează pe parametrii definiți în `constants.py`, optimizați pentru forma feței fiecărui personaj:

Personaj	**Dimensiune Fereastră** ($H \times W$)	**Prag de Detecție (Threshold)**
Fred	66×48 pixeli	0.65
Daphne	50×41 pixeli	0.90
Shaggy	62×39 pixeli	0.87
Velma	50×50 pixeli	0.85
Unknown	41×28 pixeli	0.50

Tabela 1: Parametrii specifici fiecărui model utilizat.

Scalare (Scales): Imaginile sunt redimensionate folosind un set extins de factori de scalare pentru a detecta fețe de dimensiuni variate. Lista scalelor utilizate este:

$$\text{SCALES} = [0.2, 0.3, 0.4, \ldots, 1.9, 2.0]$$

Pasul de glisare (Stride): S-a utilizat un stride de **16 pixeli**. Acest pas reprezintă un compromis optim între viteza de execuție și granularitatea detecției.

3.2 Fluxul de procesare

Pentru fiecare imagine de test:

1. Se iterează prin fiecare scală din lista `SCALES`.
2. Se redimensionează imaginea conform scalei curente.
3. Pentru fiecare personaj, se generează ferestre de dimensiunea specifică ($H \times W$).
4. Ferestrele sunt trecute prin modelul CNN (în batch-uri de 256 pentru eficiență).
5. Se aplică funcția `sigmoid` pe output și se păstrează doar detecțiile care depășesc **threshold-ul** specific clasei.
6. Coordonatele ferestrelor detectate sunt transformate înapoi în spațiul imaginii originale (împărțind la factorul de scalare).

3.3 Post-procesare: Non-Maximal Suppression (NMS)

Deoarece metoda sliding window generează multiple ferestre suprapuse pentru aceeași față, am aplicat algoritmul **Non-Maximal Suppression**. Acesta elimină detecțiile redundante pe baza scorului de suprapunere (IoU), păstrând doar fereastra cu scorul de încredere cel mai mare.

4 Rezolvarea Task-urilor

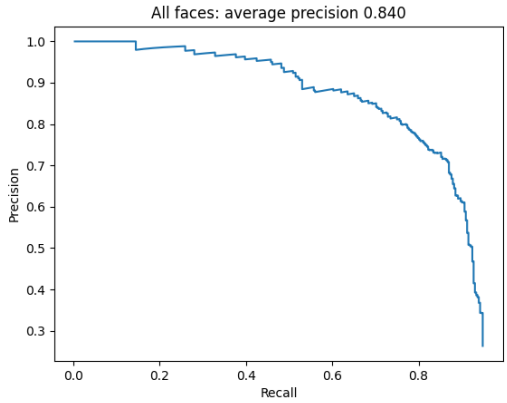
Task 2 (Recunoaștere Facială): Pentru acest task, am rulat modelele specifice celor 4 personaje principale (Fred, Daphne, Velma, Shaggy). Fiecare model a generat fișierele `.npy` necesare (`detections_fred`, etc.), conținând doar detecțiile pentru clasa respectivă care au trecut de pragul de confidență ridicat și de NMS.

Task 1 (Detectare Facială): Pentru detectarea tuturor fețelor, am adoptat o abordare de tip ansamblu:

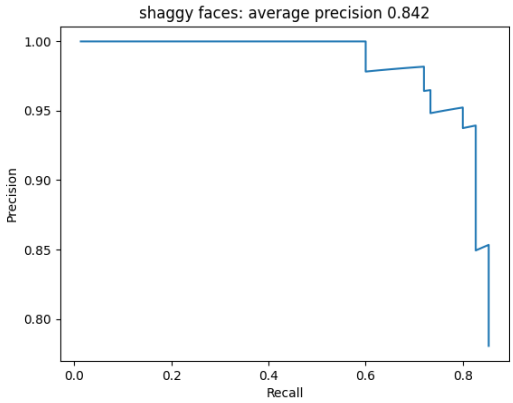
$$\text{Detections}_{all} = \text{Detections}_{Fred} \cup \text{Detections}_{Daphne} \cup \cdots \cup \text{Detections}_{Unknown} \tag{1}$$

Am concatenat rezultatele tuturor celor 5 modele. Modelul "Unknown" a fost antrenat pe un set divers de fețe (inclusiv personaje secundare), având un prag mai permisiv (0.5), asigurând astfel un Recall ridicat pentru fețele generice, în timp ce modelele specializate asigură o precizie mare pentru protagoniști.

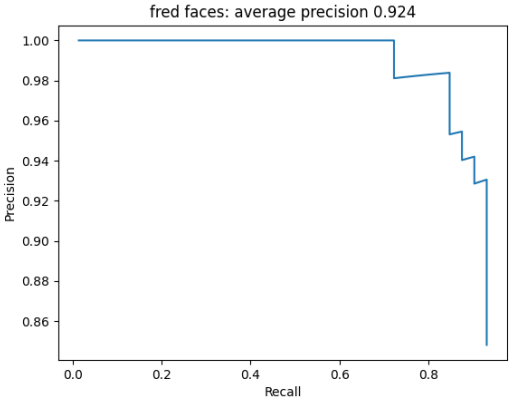
5 Rezultate



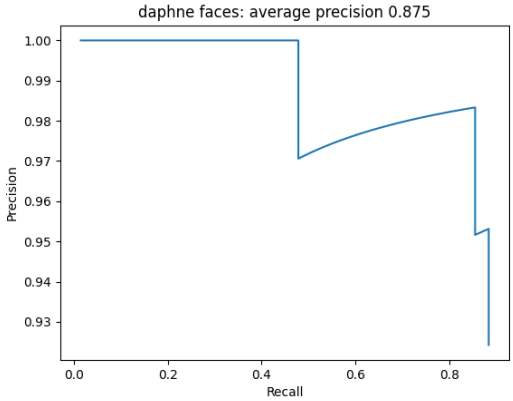
(a) All faces



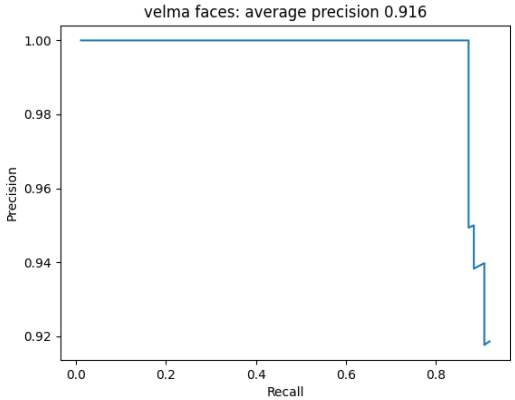
(b) Shaggy



(c) Fred



(d) Daphne



(e) Velma

Figura 1: Rezultate CNN pentru toate personajele

6 Soluție Bonus: Detecție cu YOLO

Pentru a obține punctajul bonus și pentru a compara performanța abordării clasice (Sliding Window + CNN) cu o metodă *State-of-the-Art*, am implementat o soluție secundară bazată pe arhitectura **YOLO** (You Only Look Once): modelul **yolov8n**.

Această abordare necesită o pregătire specifică a datelor, deoarece YOLO impune un format diferit de adnotare față de cel furnizat în enunțul temei.

6.1 Pregătirea Setului de Date (Data Preprocessing)

Scriptul de preprocesare implementat realizează conversia adnotărilor din formatul original (coordonate pixel: $x_{min}, y_{min}, x_{max}, y_{max}$) în formatul normalizat cerut de YOLO ($x_{center}, y_{center}, width, height$).

Procesul automatizat include următorii pași:

1. **Maparea Claselor:** S-a definit un dicționar pentru a asocia fiecărui personaj un index numeric unic, necesar pentru antrenarea clasificatorului din YOLO:
   - 0: Daphne, 1: Fred, 2: Shaggy, 3: Velma, 4: Unknown.
2. **Normalizarea Coordonatelor:** Coordonatele au fost transformate relativ la dimensiunile imaginilor ($W = 480, H = 360$).
3. **Structurarea datelor:** Scriptul împarte automat setul de date în două submulțimi: **train** (80%) și **val** (20%).
4. **Gestionarea Fișierelor:** Deoarece numele fișierelor se repetă în directoarele originale scriptul redenumește imaginile la copiere pentru a evita suprascrierea.

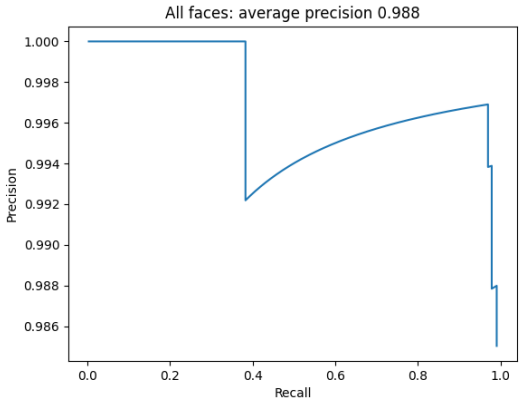
6.2 Configurarea și Antrenarea Modelului

După generarea structurii de directoare `images/train`, `images/val` și a etichetelor corespunzătoare în `labels/`, am antrenat modelul YOLO folosind acest dataset custom.

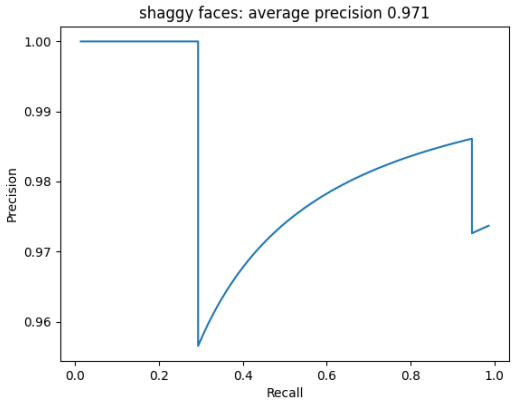
Avantajele utilizării YOLO față de soluția Sliding Window implementată la Task 1 includ:

- **Viteza de inferență:** YOLO procesează întreaga imagine într-o singură trecere prin rețea, eliminând necesitatea de a extrage mii de ferestre și de a le clasifica individual.
- **Context Global:** Rețeaua vede întreaga imagine, reducând numărul de detecții false în zonele de fundal ("False Positives").
- **NMS Integrat:** Arhitectura include mecanisme optimizate de suprimare a non-maximelor direct în procesul de predicție.

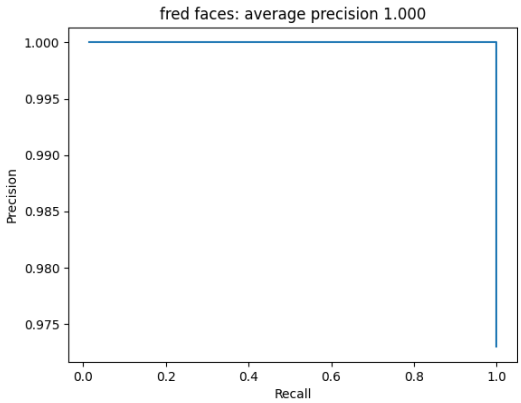
Acest rezultat a fost capabil să localizeze și să clasifice personajele simultan, ajungând la 98.8% AP (Task 1) și peste 95.2% mAP (Task 2).



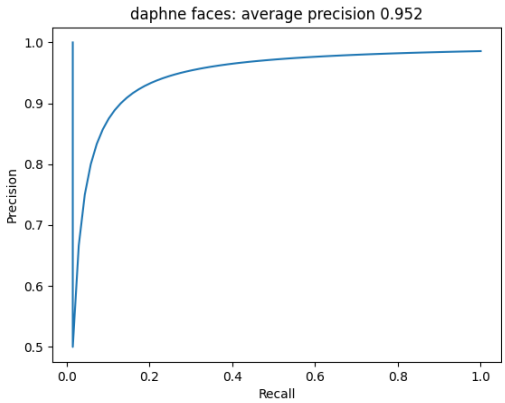
(a) YOLO All faces



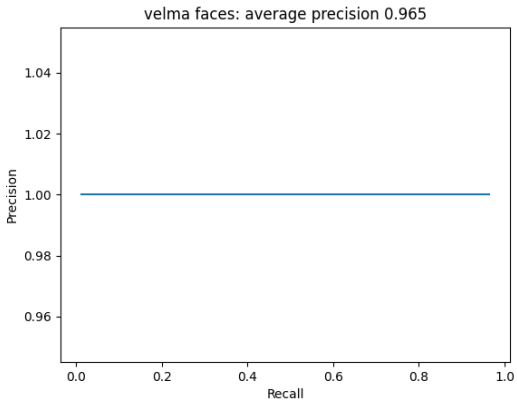
(b) YOLO Shaggy



(c) YOLO Fred



(d) YOLO Daphne



(e) YOLO Velma

Figura 2: Rezultate YOLO pentru toate personajele

7 Concluzii

Soluția propusă utilizează o arhitectură robustă de tip CNN antrenată cu *Focal Loss* pentru a combate dezechilibrul claselor. Performanța sistemului este maximizată prin ajustarea dimensiunilor ferestrelor și a pragurilor de decizie pentru fiecare personaj în parte, permițând obținerea unor rezultate bune atât la precizie, cât și la recall. În comparație, arhitectura YOLO (You Only Look Once) oferă o viteză de inferență mult mai mare, procesând contextul global al imaginii într-o singură trecere, spre deosebire de scanarea secvențială specifică ferestrelor glisante.